

Detokenization Leaks: Reconstructing Local LLM Outputs From Cache Traces

Roy Weiss
Ben Gurion University
Israel

Benyamin Konstantinov
Ben Gurion University
Israel

Eitam Sheetrit
Microsoft Security
Israel

Tomer Simon
Microsoft Security
Israel

Yisroel Mirsky*
Ben Gurion University
Israel

Abstract

We present a new attack that reconstructs the text generated by locally hosted LLMs by observing CPU cache activity during detokenization. Unlike prior attacks that rely on deployment-specific assumptions, such as shared data memory, CPU offloading, or Mixture-of-Experts architectures, our approach targets the detokenizer, a component used in default LLM inference pipelines. To obtain clean signals, we use Flush+Reload on shared tokenizer code to detect when decoding occurs, which lets us perform Prime+Probe at the right moment and isolate token-dependent cache activity. We then apply a clustering-and-language-model pipeline to recover text from noisy cache observations. We evaluate the attack across multiple datasets, hardware platforms, inference frameworks, and model families, and show that it can recover semantically accurate outputs from real-world local LLM deployments, including agentic systems.

This vulnerability is particularly significant because the most widely used tokenizer implementations are susceptible to the attack and are embedded in many popular local LLM products and agent frameworks, including systems such as OpenClaw (which we demonstrate), substantially broadening the practical attack surface.

CCS Concepts

• Security and privacy;

Keywords

Large Language Models, Side-Channel Attacks

ACM Reference Format:

Roy Weiss, Benyamin Konstantinov, Eitam Sheetrit, Tomer Simon, and Yisroel Mirsky. 2026. Detokenization Leaks: Reconstructing Local LLM Outputs From Cache Traces. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 18 pages. <https://doi.org/XXXXXXX.XXXXXXX>

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/08
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large Language Models (LLMs) have rapidly transformed sectors such as healthcare, finance, education, and entertainment. Models such as ChatGPT and Llama provide high-quality natural language generation, summarization, and decision support across diverse applications. Increasingly, users rely on LLMs for sensitive tasks, including preliminary medical advice, drafting confidential communications, and financial planning [44]. Beyond standalone usage, LLMs are now embedded in autonomous or semi-autonomous agent systems that orchestrate multi-step workflows, interact with external tools, and manage user data over extended sessions. Frameworks such as OpenClaw and Claude-Code enable agents to perform tasks including file management, coding, and personal assistance, often operating on sensitive inputs such as local files, credentials, and private communications [13, 56].

A Shift to Local Hosting. As LLM adoption grows, there is a clear trend toward local deployment, driven by privacy, latency, and reduced reliance on external services. Advances in quantization, pruning, and optimized runtimes have made local execution feasible on consumer hardware [14, 30, 39]. Tools such as Llama.cpp [20], GPT4All [3], and Ollama [49] allow users to run models including Llama [21], Mistral [28], and Phi [1] on desktops and laptops with minimal setup. This paradigm extends to agent-based systems, where users deploy LLM-powered agents locally to retain control over data and execution [42]. These agents typically run as long-lived processes with persistent access to local resources (e.g., files, APIs, and credentials), and therefore process sensitive information continuously over extended interactions.

Side-Channel Attacks on Local LLMs. While local deployment improves privacy by keeping data on-device, it also shifts the threat model toward the underlying hardware. In particular, co-resident processes on the same machine may observe low-level microarchitectural side effects of LLM inference. One important class of such leakage arises from the CPU cache: because programs access memory through shared cache structures, an adversarial process can monitor cache contention to infer which memory locations another process is accessing. Since these memory access patterns depend on the data being processed, they can inadvertently reveal sensitive information. This creates an opportunity for adversaries to extract information directly from local LLM executions.

A Lack of Universality in the Threat Model. Recent works have demonstrated cache-based side-channel attacks that exploit this effect to infer text generated by LLMs. However, these attacks rely on strong, deployment-specific assumptions. They require particular

system or model features, such as unified CPU/GPU memory [2], CPU layer offloading [18], or the use of a Mixture-of-Experts (MoE) model architecture [16]. Although these conditions can arise, some are rare, and none are *universally* present across LLM deployments. For example, memory sharing of internal model data depends on specific system configurations, and many deployments use standard (non-MoE) architectures.

Proposed Approach. In this paper, we propose a more broadly applicable cache-based side-channel attack, one that makes far fewer assumptions about the victim’s LLM libraries and configuration. Our attack is stronger because it targets a component that is *universal* to all LLM pipeline deployments in their default configuration: the detokenizer. LLMs read and generate tokens, which are akin to words or word chunks. When a token is generated by the model, it is represented as an integer ID. To read the text, each ID must be mapped back to its corresponding word chunk. This occurs on the CPU via a lookup table, such as a hash map [6, 8]. Therefore, each time a token is generated, we expect to see a unique pattern in the cache that can be traced back to each token.

Improved Universality. To avoid relying on strong assumptions such as shared data memory, we instead employ a Prime+Probe attack to observe memory access patterns in the CPU cache. However, Prime+Probe is inherently noisy and is typically effective only when memory access patterns either repeat frequently or involve large memory regions that produce strong, distinguishable signals. In our setting, neither condition holds: each token’s decode operation touches only a small portion of the cache, resulting in weak and highly ambiguous signals that are difficult to isolate and interpret.

To address these challenges, we rely on two key insights that enable the effective use of Prime+Probe in this setting:

Insight 1. *Precise timing enables cleaner signals.* If we can align Prime+Probe measurements tightly around the detokenization step, we can significantly reduce noise and isolate the cache activity of interest. We achieve this using an auxiliary side channel: specifically, we apply Flush+Reload to the tokenizer library. In common deployments, tokenizer implementations (e.g., in `Llama.cpp`) are loaded as shared libraries by the operating system.¹ When the tokenizer code is invoked, it triggers a detectable signal, allowing us to precisely time our Prime+Probe measurements around each token decode operation.

Insight 2. *Cache collisions are inevitable but structured.* Even with improved timing, cache signals remain noisy, and different tokens may map to overlapping cache sets, making them difficult to distinguish directly. To address this, we group tokens based on similarity in their observed cache access patterns and assign each group a discrete symbol. This produces a reduced symbolic representation of the trace sequence. Because these symbols preserve the order of generated tokens, we then train a downstream language model to map these symbol sequences to text, leveraging linguistic context to resolve ambiguities and reconstruct the original output.

A Widespread Vulnerability. Focusing on the tokenizer exposes a broad and practical attack surface. In current LLM ecosystems, a

¹In contrast to [2, 18], which assume shared data structures (often unavailable or disabled in standard deployments), we rely only on shared instruction pages (shared libraries), shared by default across processes in modern operating systems [64].

small number of tokenizer implementations (primarily `Llama.cpp` [20] and HuggingFace Transformers [58]) are reused across a wide range of applications. As illustrated in Fig. 1, these libraries underpin numerous locally hosted systems, including LLM servers and agent frameworks, all of which rely on them as backend components. Since these local instances run long-term, an adversary with minimal privileges can use this attack to (1) profile a target instance, (2) learn the mapping between cache traces and tokens using our method, and (3) subsequently reconstruct future outputs from that instance. This enables the extraction of sensitive user information across a diverse set of real-world deployments.

In our evaluation, we show that the attack can decipher content from a wide range of LLM tasks, from conversational LLMs to instruct LLMs to code LLMs, using three datasets (UltraChat [15], ChatDoctor [32], and Code-Alpaca [7]). We also evaluate our attack across two hardware platforms (laptop and desktop), two inference frameworks, and two target models, covering diverse tokenizer implementations and application domains. We also evaluate an end-to-end attack on the popular locally hosted agent OpenClaw [42].

Contributions. In summary, this paper makes the following contributions:

- **A More Universal Cache-Based Side-Channel Attack.** We present a detokenization-based side-channel attack that operates under default system configurations, avoiding the strong assumptions required by prior work (e.g., shared data structures, specific model architectures, or specialized CPU/GPU memory configurations). This significantly broadens the applicability of cache-based attacks on local LLM deployments.
- **Precise Prime+Probe via Auxiliary Synchronization.** We show how to obtain fine-grained temporal alignment for Prime+Probe by leveraging Flush+Reload on shared tokenizer code. This enables targeted measurement of detokenization events and substantially improves signal quality despite the inherent noise of Prime+Probe.
- **Robust Trace-to-Text Reconstruction from Noisy Signals.** We introduce a novel pipeline for mapping noisy, collision-prone cache traces to natural language. Our approach clusters continuous timing data into symbolic sequences and uses language models to reconstruct text, leveraging context to resolve ambiguity.
- **Comprehensive Evaluation on Real-World Systems.** We evaluate our attack across diverse LLM tasks, hardware platforms, tokenizer frameworks, and model families, and demonstrate end-to-end attacks on real-world deployments, including a popular locally hosted agent system (OpenClaw).

2 Background

In this section, we review the LLM inference pipeline and the cache side-channel primitives relevant to our setting, focusing on decode-time behavior and shared CPU resources. These components together define the attack surface exploited in our setting.

2.1 LLM Inference and Tokenization

LLMs operate on discrete units of text known as *tokens*, which typically correspond to words, subwords, or characters. During inference, the model consumes a user-provided input prompt and

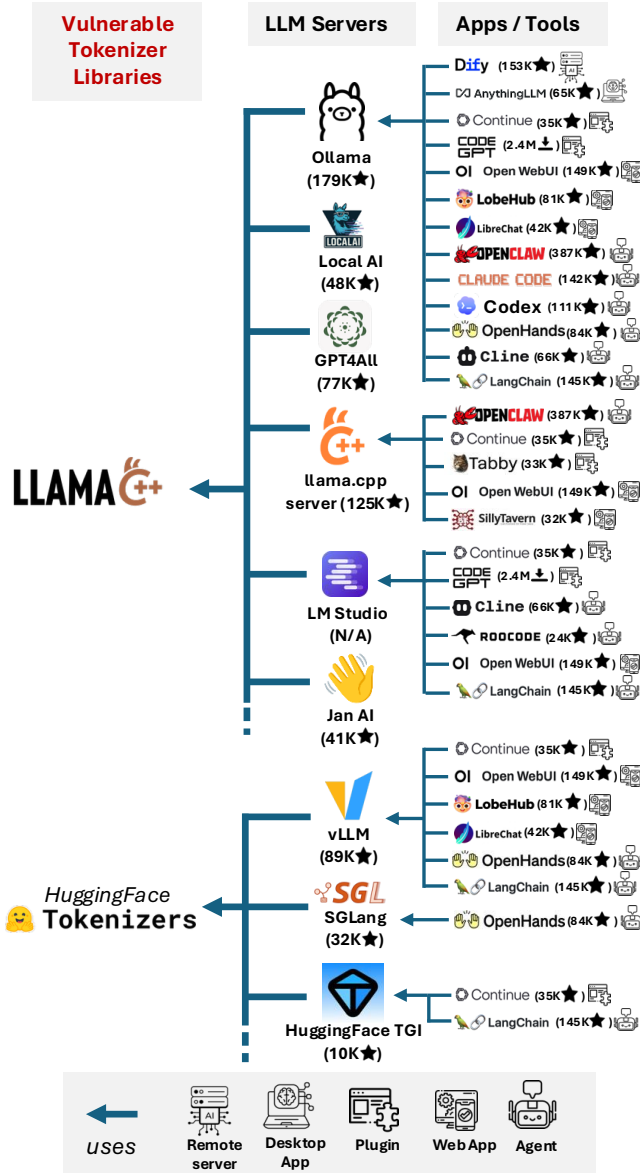


Figure 1: Tokenizers such as Llama.cpp and HuggingFace Tokenizers are vulnerable to our attack. Because these libraries are widely deployed in local LLM servers, the vulnerability extends to a large ecosystem of downstream applications, tools, and IDE plugins that depend on them (as of August 2026).

generates a sequence of output tokens in an autoregressive manner. This process involves two distinct stages: the *encode* phase and the *decode* phase.

In the *encode* phase, the input prompt string $p \in \Sigma^*$ is tokenized on the CPU into a sequence of token IDs $(p_1, p_2, \dots, p_n) \in V^n$, using a deterministic vocabulary (V) defined by the tokenizer. These IDs are then passed to the model as the initial inference context.

The *decode* phase maps each token ID to its corresponding string. At each step, the model predicts the next token ID r_i based on the previously generated context. To present the output in human-readable form, these integer token IDs are converted to UTF-8 strings using the *decode table* (D), which resides on the CPU, in contrast to the model parameters that typically remain on the GPU. To provide real-time feedback, inference frameworks invoke the decode operation immediately after each token is generated, making this mapping process inherently autoregressive.

2.2 Local LLM Deployments

Advances in quantization, pruning, and optimized inference runtimes have enabled LLMs to run directly on commodity personal devices [14, 30, 39]. Motivated by latency and privacy, local deployments are increasingly used in developer tools, document processing systems, and personal assistants [60, 66, 69].

Most local deployments follow a client-server architecture in which an inference backend exposes a localhost API to desktop interfaces, IDE plugins, or automation tools. Clients submit prompts and receive responses, typically streamed token by token as they are generated and decoded. To avoid repeatedly loading model weights, the backend remains active and serves multiple requests over time. Systems such as Ollama and LM Studio follow this design [49].

2.3 Cache Side-Channel Attacks

Modern CPUs employ caches to reduce memory access latency. These caches are organized into multiple levels, typically L1, L2, and the Last-Level Cache (LLC). L1 and L2 caches are private to individual cores, while the LLC is shared across cores. On each memory access, the CPU first checks the cache hierarchy: a cache *hit* returns data quickly, while a *miss* fetches it from main memory and stores it in the cache. This shared caching behavior creates observable contention between processes, enabling an attacker to infer a victim’s memory access patterns from a separate process.

Flush-based Attacks. Flush-based side-channel techniques, such as Flush+Reload [64] and Flush+Flush [22], exploit scenarios where memory pages are shared between an attacker and a victim. In Flush+Reload, the attacker evicts a shared cache line using `clflush` and measures the time to reload it; a fast access indicates that the victim accessed the same line.

This enables high-resolution monitoring of specific cache lines corresponding to code or data. In practice, shared memory arises from shared libraries (e.g., dynamically linked code). Extending this to arbitrary data structures requires additional conditions, such as explicit shared mappings, page deduplication, or unified memory. As these are not consistently present, the applicability of Flush+Reload-style attacks on data is limited.

Prime+Probe Attacks. Prime+Probe is a contention-based side-channel technique that does not require any shared memory between attacker and victim. Instead, it exploits the fact that multiple addresses may map to the same cache set, creating measurable interference. The attacker first fills selected cache sets with their own data (prime phase), allows the victim to execute, and then measures access times to the same addresses (probe phase). Increased access latency indicates that the victim has evicted some of the

attacker’s cache lines, revealing information about the victim’s memory accesses.

L1 Prime+Probe. In our setting, we focus on the L1 data cache, which is private to a physical core but shared between simultaneous multithreading (SMT) siblings. This enables an attacker co-scheduled on the same physical core to monitor L1 cache activity with high temporal fidelity and low measurement latency, making it well-suited for short per-token decode operations in autoregressive LLM inference. On common x86 systems, L1 set selection is determined by the virtual address bits within the page offset, so accesses to different entries may contend on the same set even without shared memory. At the same time, L1 Prime+Probe is challenging: the cache is small, the hit-miss timing gap is narrow, and the number of sets is limited. As a result, measurements are noisy and multiple memory locations may alias to the same observed cache set, reducing spatial resolution [45].

2.4 Decode Tables in Practice

The *decode table* D is a data structure that maps token identifiers to their corresponding UTF-8 byte sequences. In mainstream implementations, it is stored as an array-like structure indexed by token ID. For example, in `Llama.cpp`, which underlies frameworks such as `Ollama` and `GPT4All`, it is implemented as a `std::vector`. Other libraries use equivalent containers. In HuggingFace’s `Tokenizers` library, for example, the decode table is implemented using Rust’s default `HashMap`. Although its randomized hash seed may cause the memory layout to differ across process invocations, the layout remains fixed once initialized. Thus, within a long-lived process, decoding the same token consistently accesses the same memory locations, producing stable and repeatable cache access patterns.

3 Attack Overview

3.1 Threat Model

Victim Setup. We consider a user running an LLM locally on a personal device, either as a standalone server or as the backend of a local application or agent framework. Such deployments are increasingly used for sensitive tasks, including summarizing emails, drafting confidential communications, providing personal advice, and managing files and services.

This setting appears in several common forms: (i) local model servers used by developer tools, (ii) desktop assistants that optionally expose a localhost API, and (iii) workstation inference services shared by multiple local clients. In practice, these deployments typically run a long-lived backend process that exposes an API accessible over localhost. For example, `Ollama` binds to `127.0.0.1:11434` by default and exposes an HTTP API such as `POST /api/generate`, through which any local process can submit prompts and receive responses. Higher-level applications and agent frameworks connect to such local endpoints. For example, `OpenClaw` supports local model providers including `Ollama` and other local inference backends.

Persistence and execution. We assume that inference runs locally on the host CPU and/or GPU and that the LLM instance is long-lived, serving multiple requests over time. This ensures that tokenizer

data structures remain stable during execution, as described in Section 2.4.

Platform assumptions. We assume the victim system employs simultaneous multithreading (SMT), such as Intel Hyper-Threading, exposing multiple logical processors per physical core. This is a practical assumption, as SMT is commonly enabled by default on consumer systems [27], including across a broad range of modern CPUs summarized in Appendix Table 7. Moreover, disabling SMT can reduce performance by 25%–35% [53], providing a practical incentive to leave it enabled. Under SMT, the adversary can identify the processor executing the victim and place its own thread on the corresponding sibling processor using standard unprivileged affinity mechanisms, as commonly assumed in SMT-based side-channel attacks [5, 19].

Finally, we assume that the tokenizer implementation includes executable code pages that are shared across processes, as is standard for dynamically linked shared libraries and native extension modules in modern operating systems. We verified this property for the popular libraries we evaluated, including HuggingFace’s `Transformers` library and `Llama.cpp` as shown in Figure 1. Our attack does not assume that tokenizer data structures are shared; it relies only on shared code pages, which are enabled by default.

Adversary Model. The adversary’s objective is to reconstruct the plaintext output generated by the hosted LLM instance, or more generally, to infer sensitive information from that output. To achieve this objective, we consider an unprivileged adversarial process co-located on the same machine as the victim and scheduled on the corresponding SMT sibling. The adversary runs as a benign user-space application (e.g., a desktop app or IDE plugin) and does not require elevated privileges or modifications to the operating system or LLM framework.

Capabilities. The adversary can (1) execute standard user-space instructions, including cache line flush (`clflush`) and high-resolution timing (`rdtsc`), (2) access shared code pages of dynamically linked libraries used by the victim, and (3) interact with the local LLM service by issuing prompts and observing outputs through its exposed local interface. None of these capabilities requires special privileges. As is standard in profiling-based side-channel attacks, the adversary can use this access to collect labeled traces and learn a mapping between cache observations and generated text before applying the attack to unknown executions [16, 67, 68, 70].

Knowledge. Our attack is agnostic to the victim’s model family, model version, quantization, vocabulary size, inference framework, and application configuration. The only implementation-specific information required is the tokenizer library used by the victim, since its shared decode routine provides the Flush+Reload trigger. This library need not be known a priori: as described in Section 4, the attacker can identify the active tokenizer implementation by monitoring candidate decode routines while inducing generation. These assumptions are consistent with prior cache side-channel threat models [34, 64].

Exclusions. The adversary does not require access to shared data structures, internal model states, or GPU memory. Unlike prior work [2, 18], we do not assume any shared data memory between attacker and victim. The attack relies only on shared instruction

pages from dynamically linked libraries, which are read-only and shared by default in modern operating systems.

3.2 Vulnerability Breakdown

Key insight. The core vulnerability is that LLM detokenization relies on a *deterministic decode table* whose memory access pattern depends on the generated token. During inference, each output token r_i is represented internally as an integer ID, and the tokenizer's decode function maps that ID to its corresponding UTF-8 string by reading the decode table D . Because different tokens correspond to different entries in D , decoding different tokens causes accesses to different memory locations. As discussed in Section 2.4, this mapping remains fixed for the lifetime of the process. As a result, the same token repeatedly induces the same cache activity, creating a stable leakage source that can be profiled and later exploited.

Why this matters. This leakage arises from a component that is fundamental to LLM inference: every generated token must eventually be converted back into text. Unlike prior attacks that depend on deployment-specific features such as shared data memory or particular model architectures, our leakage source comes from the decode-table lookup itself, making the attack applicable across a broad range of local LLM deployments.

Technical formulation. As discussed in Section 2.1, LLMs operate over a fixed vocabulary of token identifiers. During generation, each predicted token r_i must be decoded into a human-readable string by querying the decode table D . Since different token strings reside at different memory locations, the memory access pattern of the decode operation depends on r_i .

Although one process cannot directly observe another process's memory accesses, each lookup of r_i induces activity in an L1 cache set c_j that can be detected via Prime+Probe. The mapping from a token identifier r_i to its cache set is given by

$$c_j = (((r_i \cdot E) \bmod P) \gg \log_2 B) \& (N - 1),$$

where E is the size of an entry in the decode table D , P is the page size, B is the cache line size, and N is the number of L1 cache sets. On typical x86 systems ($B = 64$, $N = 64$), this simplifies to

$$c_j = ((r_i \cdot E \bmod P) \gg 6) \& 63.$$

As detailed in Section 2.4, the decode-table layout remains fixed after initialization, so the mapping from a token identifier r_i to its corresponding cache set c_j remains stable during execution. This enables a profiling-style attack: the adversary collects a limited set of *labeled* traces from the same long-lived victim by issuing benign queries and observing the corresponding outputs, producing trace-token pairs that capture decode-table accesses. These pairs are then used to learn a mapping from cache traces to tokens and reconstruct outputs generated for other applications or users of the same server.

Let $t_i \in \mathbb{N}^N$ denote the cache trace collected during the decode step of token r_i , where each component records the access latency of one L1 cache set. The trace t_i exhibits a token-dependent signature, with its dominant signal at index c_j . Observing the sequence $(t_1, t_2, \dots)$ therefore reveals information about the ordered token sequence $(r_1, r_2, \dots)$, enabling reconstruction of future outputs from traces alone.

Insight. Detokenization exposes an output-aligned side channel. Each generated token is decoded in output order, so the resulting cache observations form a noisy representation of the generated token sequence itself.

However, exploiting this leakage is non-trivial due to several inherent challenges associated with Prime+Probe:

Challenge 1 (C1): Cache Collisions. The L1 cache of a consumer CPU has only about 64 cache sets that an attacker can monitor with Prime+Probe. Yet each set may correspond to an enormous number of memory addresses. For example, on a system with 8GB of RAM, over 100 million addresses can map to the same set. This massive collision space makes it extremely difficult to isolate and reliably monitor a single address. As a result, naively applying Prime+Probe to monitor D is impractical.

Challenge 2 (C2): Low signal-to-noise ratio. In the L1 cache, the timing difference between a hit and a miss can be only a few cycles, making measurements highly sensitive to noise. OS activity and scheduling interference further reduce signal clarity. Since our attacker operates in a single-shot setting (the generated token keeps changing), averaging across repeated measurements (as done in other works) is not possible.

Challenge 3 (C3): Coarse spatial resolution. The number of L1 cache sets N is small relative to the vocabulary size $|V|$. Because Prime+Probe observes activity only at cache-set granularity, the mapping from tokens to sets is many-to-one. Multiple tokens may therefore map to the same cache set, creating aliasing and ambiguity in reconstruction.

To make Prime+Probe effective despite these limitations, we address **C1** by performing Prime+Probe at the moment a token is about to be decoded. By narrowing the probe window, we eliminate a significant number of collisions that occur due to other processes. We accomplish this with an auxiliary side channel based on Flush+Reload, which is used to detect when the victim invokes the decode function, upon which the spy process triggers a Prime+Probe measurement to capture the resulting cache activity (§4.2). To address **C2**, we characterize the timing behavior of the decode routine and use it to choose when probing begins and the delays between Prime+Probe steps. This improves overlap with the victim's token-dependent accesses and yields more stable measurements (§4.3). Finally, to mitigate the ambiguity introduced by **C3**, we employ a language model-based reconstruction method that translates noisy cache traces into semantically meaningful token sequences (§4.4).

3.3 Attack Workflow

At a high level, the attack has four steps: (1) wait for the LLM to be prompted, (2) detect when the next generated token is about to be decoded using Flush+Reload (Section 4.2), (3) collect cache traces for the current token using Prime+Probe (section 4.3, repeat steps 2-3 for the entire response), (4) translate traces into text using a predictive model (Section 4.4).

To realize this attack, the adversary must first obtain the predictive model used for reconstruction. We therefore divide the attack into two phases, illustrated in Figure 2: a *profiling* phase, in which the adversary learns the mapping from cache traces to text for a

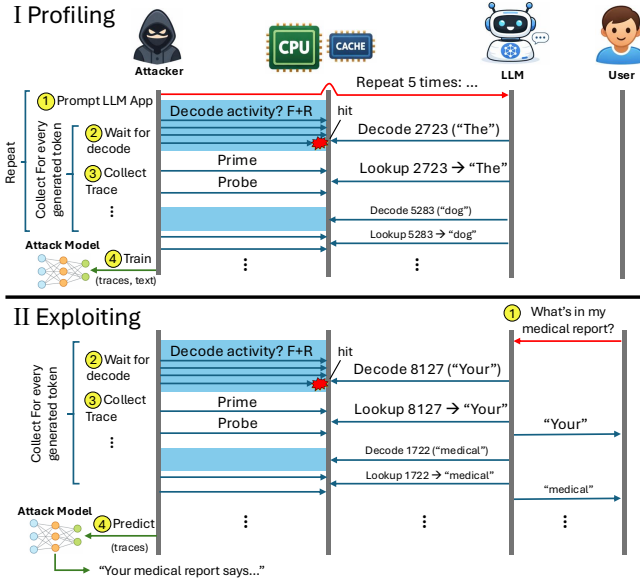


Figure 2: Attack workflow illustrating the profiling and exploitation phases and their 4 steps. During profiling, the adversary induces generation to collect traces covering common tokens and trains a predictive model; during exploitation, it uses this model to reconstruct future conversations.

specific long-lived server instance, and an *exploitation* phase, in which the learned model is used to reconstruct future outputs from the same instance.

Phase I: Profiling. The adversary first interacts with the local LLM service through its exposed API and issues its own prompts, causing the model to generate tokens whose ground-truth text is known to the adversary. In parallel, the adversary monitors decode activity using Flush+Reload and collects a cache trace using Prime+Probe for each generated token. This produces a labeled dataset that maps generated tokens to their corresponding cache traces. The adversary then trains a reconstruction model on this dataset. As shown in Section 5.4, profiling can be performed rapidly using only 250 targeted queries, or using approximately 7,650 ordinary conversational queries to obtain an equivalent profile when less distinctive interactions are desired.

Phase II: Exploitation. After profiling, the adversary passively monitors the same server instance while it serves other users or applications. The adversary again uses Flush+Reload to detect decode invocations and, on each detection, performs a Prime+Probe measurement to capture token-level cache activity. Repeating this over the full response yields a sequence of cache traces, which is then fed to the trained reconstruction model to recover the generated text.

In the next section, we will carefully detail how we designed and implemented Steps 2–4.

4 Attack Implementation

4.1 LLM Prompting (step 1)

During exploitation (Phase II), the attacker simply proceeds to Step 2 and waits for a user or application to prompt the target LLM server. During profiling (Phase I), the attacker instead actively queries the server to obtain traces paired with ground-truth response tokens. We consider two profiling approaches. In *targeted profiling*, the attacker asks the model to repeat selected token sequences, providing controlled token coverage and multiple traces per token with few queries. For example, we use queries of the form “repeat the following sequence 5 times: (sequence of 10 tokens)”. Alternatively, in *benign profiling*, the attacker issues ordinary conversational queries and collects traces from the tokens that naturally occur in the responses. As shown in Section 5.4, targeted profiling achieves strong reconstruction with only 250 queries, while benign profiling reaches comparable performance using a larger number of ordinary interactions.

4.2 Detecting an Imminent Token (step 2)

In this step, we determine when a token is about to be decoded using D in order to probe the cache at the right moment. We do so by detecting when the decode routine is executed. Building on prior work [64], we use an auxiliary side channel based on the Flush+Reload technique over shared libraries. Since tokenizer implementations are loaded as shared libraries, their instruction pages are shared across processes, enabling reliable detection of decode execution across all evaluated frameworks (Figure 1).

To set up this ‘trigger’ side channel, the adversary first identifies the tokenizer library used by the victim and the address of its decode routine. This does not require knowledge of the victim’s model or inference configuration. Common tokenizer implementations are typically open-source or publicly distributed and loaded through shared executable pages, as is standard on modern operating systems. The adversary can therefore monitor candidate decode routines while issuing prompts to the local service; the routine whose Flush+Reload activity correlates with generated output reveals the active tokenizer implementation. Once identified, the routine can be analyzed offline using standard tools such as `gdb` or `readelf`. For example, in `Llama.cpp`, the adversary may locate the symbol `llama_detokenize` at a fixed offset within the shared object file.² The adversary then maps the shared library into its own address space (e.g., via `mmap` or `dlopen`) to obtain the base virtual address, computes the target address, and monitors it using Flush+Reload. Concretely, the attacker repeatedly flushes the target cache line using `clflush` and measures its reload time; when the victim executes the decode function, the line is brought back into the cache, resulting in a significantly lower access latency that serves as a trigger signal. This process is unaffected by ASLR since the relevant code remains *physically shared* across processes.

We implement the detection phase using a widely adopted side-channel toolkit [63]. The target address is monitored in a tight loop with minimal pauses to ensure high temporal resolution. Since Flush+Reload operates at cache-line granularity, the signal remains localized, improving the signal-to-noise ratio [45, 64].

²This holds for version 0.3.16 of the `Llama.cpp` Python binding library, available at <https://github.com/abetlen/llama-cpp-python>

Experiment. To evaluate the detection performance, we simulate decode invocations at random intervals of 10–30 ms, reflecting typical token-generation delays. Over 1.2 million invocations, an attacker process continuously monitors the decode function and achieves a 97.66% true positive rate with zero false positives, demonstrating high reliability.

4.3 Capturing a Token’s Cache Trace (step 3)

If a decode event is detected from step 2, a token is *about* to be decoded. At this point, the attacker captures a trace of the decodable lookup in the cache. Since the decode table itself is not shared between processes, Flush+Reload cannot be used to monitor its token-dependent accesses. We therefore use Prime+Probe.

When the decode of token r_i is detected, we use Prime+Probe to monitor all $N = 64$ sets of the L1 cache, obtaining one trace ($t_i \in \mathbb{N}^{64}$) of access latencies. Although the attacker can reliably detect the invocation of the decode function, the precise timing of the table lookup within D remains unobservable. Therefore, through an offline analysis, the adversary optimizes the Prime+Probe measurement using high-resolution `rdtsc` timing. In our desktop `Llama.cpp` setup, for example, we found that the ideal time window is to probe from $8 \mu\text{s}$ after the trigger event for a total of $15 \mu\text{s}$. This delay remains stable across models within the same framework, such as Phi and Llama-3.

We also experimented with applying Prime+Probe to the LLC, but found it impractical for this leakage source. Unlike the 64-set L1 cache, the LLC is orders of magnitude larger and substantially slower to probe [34, 45]. Moreover, the decode table spans a large memory region whose entries are distributed broadly across the LLC, requiring the attacker to monitor a large number of cache sets. Covering this set space within the short decode window was therefore not feasible.

Figure 3 shows representative single probe traces for two tokens, illustrating how different decode operations induce distinct cache-set traces.

We briefly note several implementation details that are important for obtaining stable measurements. We construct eviction sets covering all L1 cache sets using virtual addresses that map to distinct indices, with each set represented by a small group of congruent addresses. L1 Prime+Probe measurements are inherently noisy due to hardware prefetching, limited cache capacity, and self-interference. We found that existing toolkits such as Mastik [63] do not provide sufficiently stable measurements in this regime. Instead, we use a modified version of CacheSC [24], which offers finer control over memory access patterns. In this implementation, eviction sets are traversed as randomized linked lists to avoid predictable access patterns and reduce both hardware prefetching effects and self-evictions, significantly improving signal quality.

Overall, this procedure yields one trace t_i per decoded token, forming the trace sequence $(t_1, t_2, \dots)$ for the entire generated response.

4.4 Text Inference via Predictive Model (step 4)

After collecting traces $(t_1, t_2, \dots)$, the adversary’s final step is to reconstruct the original response tokens $(r_1, r_2, \dots)$. This is challenging because thousands of tokens are compressed into just 64

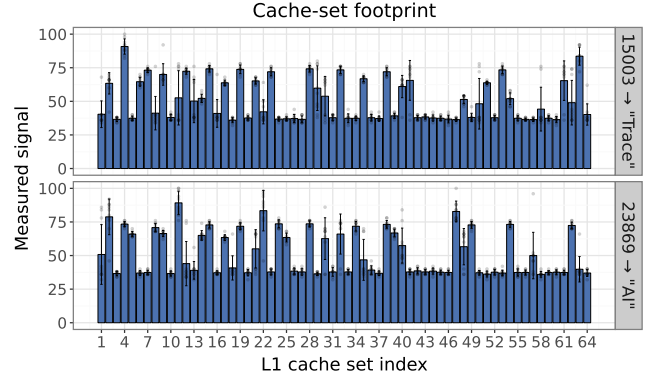


Figure 3: Example cache traces collected during the decode operation of two tokens. Each bar shows the average access latency of a cache set over 10 repetitions.

observable cache sets, leading to severe collisions that make direct trace-to-token recovery infeasible (C3). The problem is further compounded by timing noise and limited measurement resolution (C1, C2). Crucially, however, these collisions are not random: they are stable and structured. Using clustering, we map groups of tokens with similar trace patterns to shared symbols. This converts the sequence of numerical traces into a symbolic sequence that serves as a coarse representation of $(r_1, r_2, \dots)$. Although lossy, this symbolic sequence preserves substantial linguistic structure, which is sufficient for us to train a language model to reconstruct the original text. An overview of our reconstruction pipeline is shown in Figure 4.

Insight. Although individual Prime+Probe traces are ambiguous because of heavy collisions, this ambiguity is reduced when traces are viewed as a sequence and constrained by the fact that they must correspond to coherent language.

We now describe how this pipeline is instantiated in practice, focusing on (i) mapping traces to symbolic tokens, and (ii) reconstructing text from the resulting symbol sequences.

4.4.1 Trace Symbolization. As a preliminary step, we evaluate whether individual traces can be mapped directly to tokens. We train a multilayer perceptron (MLP) classifier to predict token identifiers from raw latency vectors. The classifier achieves very low accuracy (e.g., below 1% top-1), indicating that individual traces provide weak token-level information due to cache-set aliasing and measurement noise. This motivates the use of an intermediate representation that groups traces with a similar structure.

Each trace t_i is represented as a 64-dimensional latency vector, one value per monitored L1 cache set. We normalize these vectors and map them to a normalized embedding space using a lightweight MLP encoder trained with a supervised-contrastive objective [29], where traces of the same token are treated as positives. We then compute one centroid per token by averaging its profiling-trace embeddings and cluster the token centroids into K groups using K-means with cosine similarity [37].

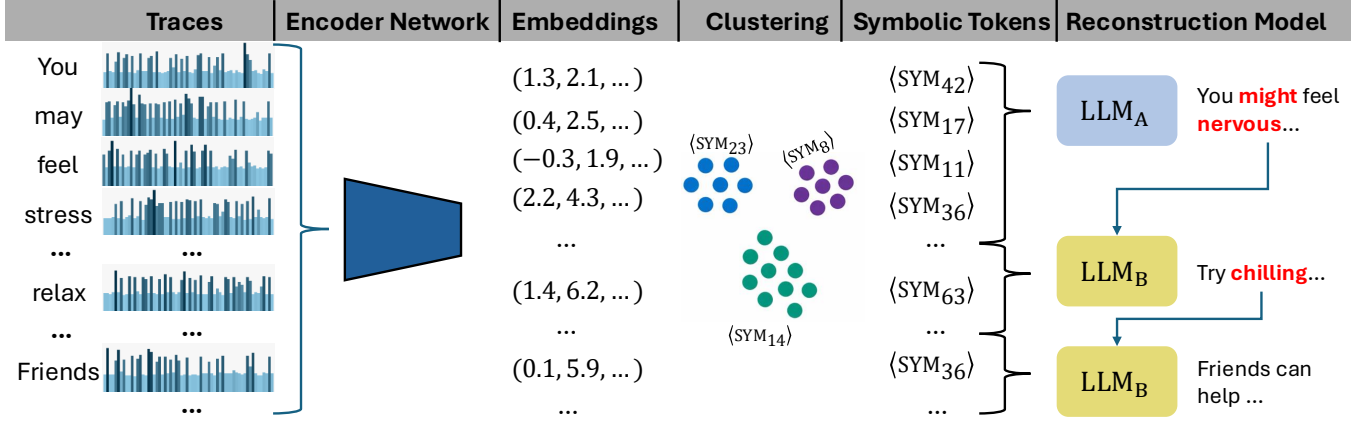


Figure 4: The trace-to-text reconstruction pipeline. Each trace in the sequence is first encoded into an embedding and assigned to a cluster learned during the profiling phase. The trace is then replaced with the symbol corresponding to that cluster, producing a sequence of symbols. Finally, this symbol sequence is passed to a language model trained to reconstruct the original tokens. Following prior work, we use separate models for the first segment (LLM_A) and subsequent segments (LLM_B).

We learn an embedding before clustering rather than applying K-means directly to raw traces, which are noisy and overlap substantially in the original latency space. Using token-level supervision available during profiling, the encoder learns to group traces of the same token and separate those of different tokens, improving cluster separation and downstream symbolization.

Each cluster defines a symbolic token $s_\ell \in S$, where $S = \{s_1, \dots, s_K\}$. During symbolization, each trace is assigned to its nearest cluster center and replaced by the corresponding symbol, producing a symbolic sequence that captures the coarse structure of the underlying token sequence. We use $K = 64$, which balances clustering fidelity and symbol diversity (Appendix F).

To integrate these symbols into the reconstruction model, we extend the tokenizer vocabulary with K synthetic entries of the form $\langle SYM_k \rangle$, one per cluster, following prior work [57]. Each is assigned a unique token ID appended to the base vocabulary, allowing symbolic sequences to be processed through the model’s standard embedding layer.

4.4.2 Semantic Reconstruction. Given a symbolized trace sequence $(s_1, s_2, \dots)$, we reconstruct the corresponding response tokens $(r_1, r_2, \dots) \in V^*$ by treating the task as sequence-to-sequence translation from symbolic tokens to natural language using Flan-T5-XL [10].

We follow Weiss et al. [57], who observe that the first sentence of LLM responses often reveals the main topic and facilitates reconstruction of subsequent content. Accordingly, they employ two separate models for staged reconstruction, a design we adopt. Our contribution instead lies in extracting informative discrete symbol sequences from noisy microarchitectural traces.

We split each response into 32-token segments $R = (R_0, R_1, \dots)$. LLM_A reconstructs the first segment R_0 from its symbol sequence alone. For $j > 0$, LLM_B reconstructs segment R_j conditioned on the symbol sequence and the previous segment text. During training, this context is the ground-truth previous segment R_{j-1} ; during inference, it is the model prediction $\hat{R}_{j-1}$.

LLM_A is prompted with the symbol sequence alone, while LLM_B is prompted with both the previous segment text and the current symbol sequence.

Both models are trained with teacher-forced cross-entropy loss. Although symbolization is many-to-one, reconstruction remains effective because the model exploits sequential structure and contextual conditioning to resolve locally ambiguous symbols [38]. Additional training details are provided in Appendix E.

5 Evaluation

In this section, we outline the evaluation setup. Then we will explore the fidelity of the reconstruction attack along three primary dimensions: generality, robustness, and cost-effectiveness.

5.1 Experiment Setup

Environment Hardware. To evaluate generality, we conduct experiments on two systems: a laptop and a desktop, both running Ubuntu 22.04 LTS. The laptop uses an Intel® Core i7-8650U CPU @ 1.90 GHz with a 32 KB L1 cache (8-way), 256 KB L2 cache (4-way), and 8 MB L3 cache (16-way). The desktop uses an 11th Gen Intel® Core™ i7-11700K CPU @ 3.60 GHz with a 48 KB L1 data cache (12-way), 32 KB L1 instruction cache (8-way), 512 KB L2 cache (8-way), and 16 MB L3 cache (16-way), and is additionally equipped with NVIDIA RTX 3090 and RTX 3080 GPUs (CUDA 12.2).

Datasets & Training. We use three publicly available LLM-generated datasets: Code-Alpaca [7], UltraChat [15], and ChatDoctor [32], covering programming, medical queries, financial advice, and general knowledge. For multi-turn datasets, we retain only the first LLM response. Each dataset is randomly split into training, validation, and test sets using an 80%/10%/10% ratio, with standard deduplication [6, 8, 31]. Further preprocessing details are provided in Appendix C.

Our evaluation additionally uses cache traces collected with the Prime+Probe procedure described in §4.3. For each token, we collect 50 traces from repeated executions of the decode routine, split into

30 profiling traces for training and 20 held-out traces for evaluation. Thus, reconstruction is evaluated only on traces unseen during profiling.

For reconstruction, we fine-tune Flan-T5-XL for 5 epochs to obtain LLM_A and LLM_B across multiple settings. Training is performed on an NVIDIA RTX 6000 Pro GPU and takes up to 6 hours for LLM_A and 4 days for LLM_B .

Metrics. We evaluate natural-language and code outputs separately, as reconstruction quality is captured differently for each. For UltraChat and ChatDoctor, we report normalized character-level Edit Distance (ED), where lower is better, ROUGE-1 F1 [33] for unigram overlap, and cosine similarity ϕ between Sentence-Transformers all-MiniLM-L6-v2 embeddings [46], where $\phi \in [-1, 1]$ and higher values indicate stronger semantic similarity. We also use GPT-4.1-mini as an LLM-based judge and report Attack Success Rate (ASR) as the fraction of samples judged semantically equivalent, following prior work on LLM-based evaluation [23, 71].

For Code-Alpaca, lexical and embedding-based metrics do not reliably capture functional correctness. We therefore use the same judge to assess whether reconstructed and reference snippets implement the same functionality, allowing benign differences such as variable renaming, formatting, or alternative correct implementations [11, 50]. Judge prompts are provided in Appendix Figure 8.

Experiments. We evaluate the full reconstruction pipeline across multiple configurations, varying the target model between Phi-3-mini [1], which uses a 32k-token vocabulary, and Llama-3 [21], which uses a 128k-token vocabulary. We further vary the tokenization framework (Llama.cpp and HuggingFace Transformers), hardware platform (laptop and desktop), and dataset (UltraChat, Code-Alpaca, and ChatDoctor). For each configuration, each of the reconstruction models is trained on a disjoint training split and evaluated on unseen traces and responses, reporting results for both first-segment and full-response reconstruction.

5.2 Attack Generality

In this section, we evaluate the attack’s performance in reconstructing both the initial segment and full paragraphs across all configurations. We distinguish between first-segment and full-response reconstruction, as the initial portion often reveals the core confidential topic.

The full results are presented in Table 1 for textual datasets and in Table 2 for code datasets, with real reconstruction examples in Figure 5. Across all settings, reconstruction achieves high semantic fidelity despite ambiguous token-level signals. For full paragraphs, UltraChat reaches ASR of ~56–87% with semantic similarity ϕ of ~76–87%, while ChatDoctor achieves substantially higher ASR of ~78–93% with ϕ of ~61–76%. Despite weak per-token signals, reconstructions often preserve subtopics and local structure even when individual words are incorrect.

This semantic robustness stems from sequence-level modeling: although individual symbols are noisy and ambiguous, the model leverages context across the sequence to resolve uncertainty, enabling accurate semantic reconstruction even when lexical similarity degrades.

To further characterize reconstruction fidelity beyond broad semantic recovery, we examine the fraction of outputs achieving very

Attack Examples - Ultrachat

Judge: Success	ϕ : 1.00	R1: 1.00	ED: 0.00
Ref: Yes, incorporating daily exercise into your routine can improve your mental health in several ways. Exercise can help reduce symptoms of depression and anxiety Pred: Yes, incorporating daily exercise into your routine can improve your mental health in several ways. Exercise can help reduce symptoms of depression and anxiety			
Judge: Success	ϕ : 0.835	R1: 0.70	ED: 0.296
Ref: There are several types of therapy that are effective for treating anxiety and depression: 1. Cognitive Behavioral Therapy (CB) Pred: There are several types of therapy that are effective for treating anxiety and depression disorders (ADHD) by combining counseling and therapy			
Judge: Success	ϕ : 0.699	R1: 0.837	ED: 0.153
Ref: Both cardio and strength training can be effective in weight loss strategies. Cardio exercises such as running, cycling, and swimming can help Pred: Cross-training and strength training can be effective in weight loss surgery. Cardio exercises such as running, cycling, and swimming often help			
Judge: Success	ϕ : 0.451	R1: 0.625	ED: 0.336
Ref: Fibromyalgia and Chronic Fatigue Syndrome (CFS) share some common symptoms, such as fatigue, sleep problems, Pred: Cyanobacteria and fungi do not always and everywhere share some common symptoms, such as fatigue, sleep problems,			
Judge: Failure	ϕ : 0.532	R1: 0.652	ED: 0.287
Ref: Studies have shown that drinking beer in moderation may have some cardiovascular health benefits, particularly for those at risk of heart disease. Pred: Studies have shown that athletes eating disorders often may have some cardiovascular health benefits, particularly at the intermediate levels of heart disease.			

Figure 5: Examples of successful and unsuccessful attacks on R_0 in UltraChat.

high semantic similarity ($\phi \geq 0.9$). For first-segment reconstruction, this rate reaches up to 62.92% on ChatDoctor and 44.62% on UltraChat. Even for full responses, up to 58.48% of UltraChat reconstructions achieve $\phi \geq 0.9$. These results show that the attack can recover outputs with high fidelity, rather than merely identifying their broad topic. Appendix G provides the full breakdown.

Insight. Semantic recovery makes the attack effective even without exact token inference, while the high-fidelity results show that the leakage can extend well beyond topic-level information to closely reconstructed outputs.

Table 1: Reconstruction performance for textual datasets, frameworks, hardware platforms, and target models. All metrics are reported as mean $\pm$ standard deviation. Higher is better for all metrics except ED ($\downarrow$).

Dataset	Framework	Hardware	Target	First Segments				Full Paragraphs			
				ASR	Φ	R1-F1	ED $\downarrow$	ASR	Φ	R1-F1	ED $\downarrow$
UltraChat	Llama.cpp	Laptop	Phi-3-mini	57.51%	69.24% $\pm$ 31.59%	77.25% $\pm$ 21.80%	77.97% $\pm$ 20.63%	85.15%	86.68% $\pm$ 14.19%	82.70% $\pm$ 9.69%	82.96% $\pm$ 9.68%
		Desktop	Llama-3	39.71%	54.53% $\pm$ 33.54%	63.99% $\pm$ 23.96%	65.72% $\pm$ 22.18%	72.03%	81.22% $\pm$ 15.67%	72.56% $\pm$ 10.28%	72.08% $\pm$ 10.37%
	HuggingFace	Laptop	Phi-3-mini	61.55%	71.62% $\pm$ 30.98%	78.72% $\pm$ 21.51%	79.35% $\pm$ 20.32%	87.42%	87.52% $\pm$ 13.80%	83.02% $\pm$ 9.54%	83.31% $\pm$ 9.56%
		Desktop	Llama-3	44.52%	59.41% $\pm$ 33.28%	69.32% $\pm$ 23.60%	70.18% $\pm$ 22.10%	74.78%	83.19% $\pm$ 15.12%	76.52% $\pm$ 10.21%	76.09% $\pm$ 10.40%
ChatDoctor	Llama.cpp	Laptop	Phi-3-mini	58.77%	69.43% $\pm$ 31.18%	75.65% $\pm$ 22.18%	77.05% $\pm$ 20.66%	86.11%	86.38% $\pm$ 14.04%	80.23% $\pm$ 9.69%	80.89% $\pm$ 9.59%
		Desktop	Llama-3	25.50%	40.08% $\pm$ 33.26%	51.18% $\pm$ 25.92%	54.21% $\pm$ 22.90%	56.68%	76.51% $\pm$ 16.99%	64.78% $\pm$ 11.48%	62.89% $\pm$ 11.60%
	HuggingFace	Laptop	Phi-3-mini	55.22%	66.58% $\pm$ 32.70%	74.21% $\pm$ 23.59%	75.40% $\pm$ 21.86%	84.04%	85.81% $\pm$ 14.51%	80.82% $\pm$ 10.17%	81.11% $\pm$ 10.14%
		Desktop	Llama-3	43.02%	57.70% $\pm$ 33.45%	67.67% $\pm$ 23.76%	68.87% $\pm$ 22.23%	73.26%	82.55% $\pm$ 15.22%	75.17% $\pm$ 10.18%	74.84% $\pm$ 10.38%
	Llama.cpp	Laptop	Phi-3-mini	84.67%	84.38% $\pm$ 23.27%	85.17% $\pm$ 19.52%	86.74% $\pm$ 17.67%	90.08%	75.25% $\pm$ 15.17%	63.89% $\pm$ 12.62%	66.28% $\pm$ 12.68%
		Desktop	Llama-3	48.67%	56.40% $\pm$ 27.75%	62.29% $\pm$ 22.67%	65.56% $\pm$ 20.50%	78.60%	67.97% $\pm$ 17.40%	66.37% $\pm$ 14.09%	67.05% $\pm$ 14.16%
	HuggingFace	Laptop	Phi-3-mini	63.89%	64.00% $\pm$ 29.22%	62.02% $\pm$ 25.86%	62.71% $\pm$ 22.43%	91.27%	74.24% $\pm$ 17.35%	70.55% $\pm$ 14.22%	71.01% $\pm$ 13.69%
		Desktop	Llama-3	51.58%	59.14% $\pm$ 28.41%	65.53% $\pm$ 23.52%	68.27% $\pm$ 21.19%	81.24%	69.74% $\pm$ 17.76%	69.08% $\pm$ 14.62%	69.60% $\pm$ 14.57%
	Llama.cpp	Laptop	Phi-3-mini	66.98%	67.62% $\pm$ 26.56%	69.12% $\pm$ 22.10%	72.44% $\pm$ 19.73%	92.85%	75.97% $\pm$ 16.54%	73.76% $\pm$ 13.10%	75.56% $\pm$ 12.65%
		Desktop	Llama-3	34.23%	47.50% $\pm$ 27.93%	52.21% $\pm$ 25.55%	57.06% $\pm$ 22.33%	59.76%	61.02% $\pm$ 17.72%	56.10% $\pm$ 16.06%	56.30% $\pm$ 16.20%
	HuggingFace	Laptop	Phi-3-mini	67.26%	68.80% $\pm$ 26.76%	72.26% $\pm$ 22.14%	74.70% $\pm$ 19.80%	92.80%	76.62% $\pm$ 16.67%	75.40% $\pm$ 13.60%	76.56% $\pm$ 13.20%
		Desktop	Llama-3	51.85%	59.59% $\pm$ 27.97%	65.95% $\pm$ 22.38%	68.38% $\pm$ 20.49%	80.61%	69.73% $\pm$ 17.61%	69.11% $\pm$ 14.26%	69.72% $\pm$ 14.30%

Table 2: Code-Alpaca reconstruction performance (ASR) for first-segment and full-output reconstruction.

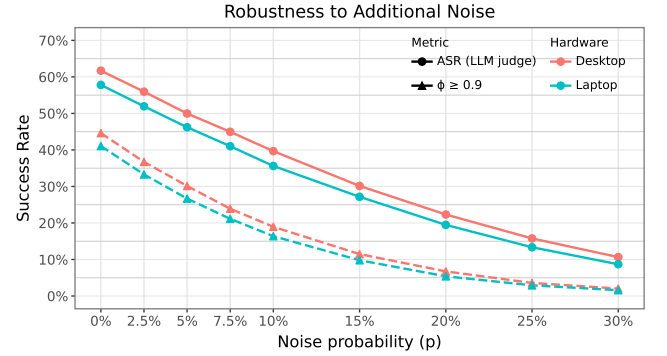
Framework	Hardware	Target	First	Full
Llama.cpp	Laptop	Phi-3-mini	90.17%	95.87%
	Desktop	Llama-3	50.13%	49.43%
HuggingFace	Laptop	Phi-3-mini	87.65%	90.56%
	Desktop	Llama-3	81.55%	86.86%
HuggingFace	Laptop	Phi-3-mini	85.19%	88.44%
	Desktop	Llama-3	64.69%	71.35%
HuggingFace	Laptop	Phi-3-mini	84.92%	89.71%
	Desktop	Llama-3	79.19%	84.72%

Performance is consistent across hardware platforms and inference frameworks, indicating that the attack does not depend on specific microarchitectural properties. Instead, it relies on stable decode-table access patterns within a given deployment, together with effective profiling to learn the corresponding mappings.

Performance differences are primarily driven by the target model and dataset. Llama consistently underperforms Phi, which we attribute to its larger vocabulary ($\times 4$) and the resulting increase in cache-set collisions. Additionally, more structured datasets such as ChatDoctor and Code-Alpaca yield higher performance than more diverse datasets like UltraChat, suggesting that constrained token distributions reduce reconstruction ambiguity.

5.3 Attack Robustness

All experiments so far use real cache traces and therefore already include natural measurement noise. To further study robustness, we inject additional synthetic noise, with probability p , into the symbolic token sequences produced by the clustering stage. Specifically, we introduce deletions, insertions, and substitutions in the clustering output. In our reconstruction pipeline, deletions correspond to false negatives in the auxiliary detection side channel, where a decode event is missed and no trace is collected. Insertions correspond to false positives, where a spurious cache hit is detected despite no true decode event. Substitutions represent clustering misclassifications, where a collected cache trace is assigned the

**Figure 6: Robustness to additional noise injected into the reconstruction pipeline. We report, for each p , the base ASR and the rate of high-fidelity predictions with $\phi \geq 0.9$ for both hardware settings.**

wrong symbolic token. This form of synthetic corruption is commonly used in prior side-channel reconstruction work to evaluate robustness to measurement errors [18, 67, 70].

For each value of p , we evaluate our base reconstruction model LLM_A , trained on standard UltraChat traces collected on the laptop using Llama.cpp with Phi-3 as the target model. The model is trained only on the original, uncorrupted traces. During evaluation, each token in the clustering output is independently corrupted with probability p , with the corruption type chosen uniformly among deletion, insertion, and substitution. This allows us to systematically measure how reconstruction quality degrades as noise increases.

As shown in Figure 6, performance gradually degrades as p increases. Even at $p = 0.15$, LLM_A still attains an ASR of $\sim 30\%$, with roughly 10% of reconstructions achieving high semantic fidelity ($\phi > 0.9$). These results show that reconstruction remains robust under substantial injected noise.

5.4 Query Performance Tradeoff

The profiling phase is critical to the reconstruction attack, as it establishes the mapping between cache activity and output tokens. In practice, profiling presents a tradeoff between the number of queries and how distinctive they are. We therefore evaluate two approaches: *targeted profiling*, which deliberately elicits selected tokens with few queries, and *benign profiling*, which uses only ordinary conversational queries at the cost of a larger budget.

We evaluate this tradeoff using the laptop, UltraChat, Llama.cpp, and Phi-3 setting. In targeted profiling, the attacker issues queries of the form “repeat the following sequence 5 times: (sequence of 10 tokens)”, obtaining five traces per selected token. Tokens are chosen by frequency in the UltraChat training split. For each targeted-query budget (Q_t), we construct an equivalent profile using ordinary UltraChat-style queries until they provide the same profiled token set and trace coverage. Figure 7 reports the resulting ASR, with the corresponding benign-query budget (Q_b) shown on the upper axis.

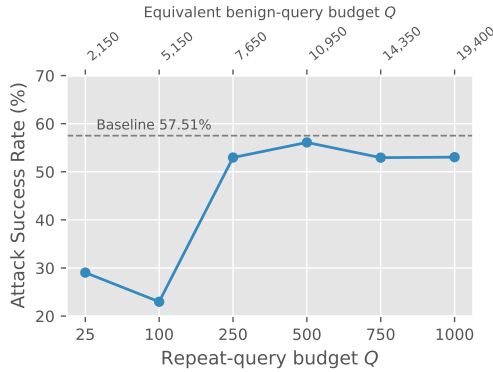


Figure 7: ASR as a function of targeted and equivalent benign profiling query budgets on Phi-3-mini.

Targeted profiling is highly query-efficient: with only $Q_t = 250$ queries, the attack exceeds 55% ASR and completes within roughly 6 minutes of standard inference on the tested hardware. Distinctive repetition queries, however, are not required. An equivalent profile can be obtained with approximately $Q_b = 7,650$ ordinary conversational queries, requiring roughly 3 hours and achieving about 53% ASR. With $Q_b = 10,950$ benign queries, ASR reaches approximately 56%, close to the unlimited-profile baseline of 57.51%. Further increases provide little additional benefit, with performance remaining around 53–56%.

These results show that profiling can be adapted to the attacker’s constraints: targeted queries enable rapid profiling, while ordinary conversational queries retain near-baseline reconstruction performance with a larger budget.

5.5 Hardware Evaluation

To evaluate the attack on recent hardware, we repeat the first-segment evaluation on a 13th Gen Intel Core i9-13950HX using Llama.cpp with Phi-3-mini and Llama-3 on UltraChat. Modern processors employ hardware prefetchers that can introduce additional

cache accesses, interfering with cache-based side-channel measurements [52, 54]. As shown in Table 3, performance degrades relative to our earlier results (Table 1), but the attack remains effective, demonstrating that the leakage persists on modern Intel hardware despite these effects.

Table 3: Reconstruction performance (first-segments) on a 13th Gen Intel CPU.

Target	ASR	$\phi > 0.5$	R1-F1	ED ↓
Phi-3-mini	38.40%	52.12%	56.97%	61.07%
Llama-3	31.97%	46.86%	53.87%	57.97%

5.6 End-to-End Attack on OpenClaw

So far, our evaluation relied on dataset-aligned traces, where cache measurements were collected and matched to pre-existing text corpora. While this isolates the reconstruction pipeline, it does not capture a fully realistic deployment. We now evaluate the attack end-to-end on a live system by performing profiling, training, and reconstruction directly on an OpenClaw agent [42] running a local Phi-3-mini model [1], without any synthetic alignment. OpenClaw is a general-purpose open-source LLM agent framework for local deployment, enabling persistent assistants that manage personal workflows and interact with local resources, and is among the most popular such systems by GitHub stars (Figure 1).

We use a subset of UltraChat prompts ($n = 25,000$) as inputs to the agent, issuing them and collecting traces during real execution. We evaluate reconstruction on the first response segments only, as these are sufficient to recover the main topic of the output, as discussed in Section 5.2. This reflects a practical deployment in which an LLM-based agent processes user queries over time.

We report reconstruction performance using ASR and mean semantic similarity ϕ . Since OpenClaw relies on Llama.cpp for local inference (Figure 1), we compare against the corresponding controlled setting (Llama.cpp, Phi, Desktop, UltraChat), which achieves 61.55% ASR and 71.62% ϕ ($\pm 30.98\%$). On OpenClaw, the attack achieves 30.12% ASR with a mean ϕ of $42.23\% \pm 20.28\%$. Performance degrades due to additional end-to-end noise, including timing variability and imperfect alignment. Nevertheless, the attack remains effective, showing that reconstruction transfers to a real-world deployment.

6 Mitigation Strategies

In this section, we discuss potential mitigations and their practical limitations in the context of our threat model.

Decode Table Randomization. As described in Section 3.1, our attack relies on a stable mapping between token identifiers and their memory footprint during the lifetime of the victim process. A natural mitigation is therefore to randomize the in-memory layout of the decode table during execution, for example, by shuffling token placements or permuting internal indexing structures. This changes the mapping between token IDs and cache sets, invalidating previously collected traces and disrupting the learned leakage model. However, an active adversary may re-profile the victim after each re-randomization event. Thus, randomization primarily

increases the attack cost rather than eliminating the underlying leakage channel.

Process Lifetime Reduction. Our reconstruction model is process-specific and does not transfer across independent instances due to ASLR, heap allocation differences, and randomized data structure layouts. Periodic restarts or short-lived workers can therefore limit the time available for profiling. However, frequent model initialization increases latency and reduces responsiveness. Moreover, this mitigation does not eliminate the leakage, as an adversary can re-profile each new instance.

Microarchitectural and OS-level Defenses. Disabling SMT, such as Intel’s Hyper-Threading [40], prevents an attacker from sharing a physical core with the victim and therefore blocks L1 Prime+Probe [51]. However, disabling SMT incurs a significant performance cost (Section 3.1), making it unattractive for resource-constrained local LLM deployments.

Cache partitioning, such as page coloring [65], and hardware mechanisms that randomize cache indexing or eviction [43] can also reduce the available signal, although profiling may partially adapt. These defenses are generally not enabled on consumer systems due to limited support and performance or deployment overhead [41], making them more suitable for multi-tenant or high-assurance environments.

7 Related Works

Privacy Attacks on Language Models. LLMs expose privacy risks at the application and network levels. Deployment optimizations such as KV-cache reuse across tokens or sessions have been exploited to infer user prompts and model outputs through fine-grained decoding timing [48] or probing shared KV-cache entries in multi-tenant settings [59]. At the network level, observing the length and structure of encrypted token streams can leak model outputs [57]. Other works show that adversarial queries can elicit hidden system prompts or reconstruct prior conversations, revealing sensitive context through repeated interaction [9, 12, 26, 62]. Together, these studies highlight non-hardware side channels that persist even under strict access isolation.

Hardware Side-Channel Attacks on Deep Learning. Hardware side channels have been used to infer neural-network inputs and outputs through power analysis, electromagnetic emanations, timing variations, or memory bus contention [35, 47]. Other studies extract model architectures and weights, including layer parameters, connectivity, and numerical values [4, 17, 25]. These attacks exploit structured, data-dependent behavior in DNN execution.

Cache-based attacks pursue similar goals. Prior work shows that cache activity can leak sensitive user inputs [67, 68, 70] or reconstruct model architectures and weights across deployment settings [36, 55, 61]. Autoregressive LLMs, however, exhibit fundamentally different execution characteristics: they generate outputs token-by-token, rely on large vocabulary lookups, and induce input-dependent, non-repetitive memory access patterns. These properties make prior techniques, which assume static or highly regular execution, difficult to apply directly.

Hardware Side-Channel Attacks on LLMs.

Table 4: Comparison of deployment assumptions across local LLM side-channel attacks. "Special" denotes non-standard shared resources, while "standard" denotes commonly shared resources in default deployments.

Attack	Model Assumptions		System Assumptions	
	Weights	Arch	SMT	Shared Resources
IKWYS [18]	CPU	None	None	Model data; special
Spill the Beans [2]	CPU	None	None	Model data; special
MoEcho-GPU [16]	None	MoE	None	GPU counters; special
MoEcho-CPU [16]	None	MoE	Yes	Expert pages; special
Ours	None	None	Yes	Code only; standard

Recent work has demonstrated hardware side-channel leakage from local LLM inference, but existing attacks rely on leakage sources that are present only under particular model or deployment conditions. Table 4 summarizes these requirements and contrasts them with ours.

One line of work exploits shared model data, particularly token embeddings. Adiletta and Sunar [2] use Flush+Reload on embedding vectors exposed through CUDA unified CPU/GPU memory to leak tokens and partially recover API keys, requiring a non-default CUDA memory configuration. Gao et al. [18] similarly monitor embedding-table accesses through shared model pages, requiring both the model data to be shared (e.g., through mmap or page deduplication) and the embedding lookup to execute on the CPU. When the model fits entirely in GPU memory, as is common in local inference, the embedding layer remains GPU-resident, and this leakage channel is absent. In contrast, our attack targets CPU-side detokenization after token generation, avoiding these model-data visibility and placement requirements altogether.

Another line of work focuses on *specific model architectures*. Ding et al. [16] show that MoE models leak expert activation patterns, enabling prompt inference and response reconstruction. Their attack variants additionally rely on specialized observability, such as GPU performance counters or shared expert data pages. More fundamentally, however, the leakage source is the MoE routing mechanism itself. As a result, the attack does not extend to standard dense models, such as common Llama-style deployments. In contrast, our attack targets detokenization, a model-agnostic step performed after each generated token.

This different leakage source also creates a substantially weaker observation channel. Prior embedding attacks observe shared data at cache-line granularity [2, 18], while MoE attacks exploit large, structured expert-execution footprints [16]. Network attacks similarly expose stable observables such as response size or latency [48, 57, 59]. A detokenization event, by contrast, is short-lived and touches few cache lines. Prime+Probe further collapses many token-dependent accesses onto the same cache sets. Individual traces therefore do not directly reveal token identities. We overcome this by mapping noisy traces to stable symbolic representations and using language context to resolve the resulting ambiguity.

Overall, prior attacks expose important leakage channels in local LLM inference, but their applicability is tied to either model-internal data exposure or architecture-specific execution behavior. To our knowledge, the decode table has not previously been examined as a leakage source. Our work shifts the attack surface to detokenization,

a default component of the generation pipeline, enabling cache-based output reconstruction across a broader range of local LLM deployments than previously demonstrated.

8 Conclusion

We presented a new cache-based attack that reconstructs the text generated by locally hosted LLMs by observing CPU cache activity during detokenization. By targeting the decode table, our attack avoids the deployment-specific assumptions required by prior work and instead exploits a leakage source that is present in default LLM inference pipelines. We showed how to combine Flush+Reload for decode detection, Prime+Probe for cache-trace capture, and a clustering-and-language-model pipeline for text reconstruction from noisy observations. Our findings identify detokenization as a practical and widespread leakage surface in local LLM serving and highlight the need for stronger isolation and defenses in privacy-sensitive deployments.

9 Acknowledgments

The authors would like to thank Lital Badash from Microsoft for her help in managing the project.

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219* (2024).
- [2] Andrew Adiletta and Berk Sunar. 2025. Spill The Beans: Exploiting CPU Cache Side-Channels to Leak Tokens from Large Language Models. *arXiv preprint arXiv:2505.00817* (2025).
- [3] Yuvanesh Anand, Zach Nussbaum, Brandon Duderstadt, Benjamin Schmidt, and Andriy Mulyar. 2023. GPT4All: Training an Assistant-style Chatbot with Large Scale Data Distillation from GPT-3.5-Turbo. <https://github.com/nomic-ai/gpt4all>.
- [4] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. 2019. {CSI}{NN}: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium (USENIX Security 19)*. 515–532.
- [5] Atri Bhattacharyya, Alexandra Sandulescu, Matthias Neugschwandtner, Alessandro Sorniotti, Babak Falsafi, Mathias Payer, and Anil Kurmus. 2019. Smother-spectre: exploiting speculative execution through port contention. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 785–800.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Charles Sutton, Sebastian Gehrmann, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [7] Sahil Chaudhary. 2023. Code Alpaca: An Instruction-following LLaMA model for code generation. <https://github.com/sahil280114/codealpaca>.
- [8] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.
- [9] Junjie Chu, Zeyang Sha, Michael Backes, and Yang Zhang. 2024. Reconstruct Your Previous Conversations! Comprehensively Investigating Privacy Leakage Risks in Conversations with GPT Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 6584–6600.
- [10] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.
- [11] Giuseppe Crupi, Rosalia Tufano, Alejandro Velasco, Antonio Mastropaolo, Denys Poshyvanyk, and Gabriele Bavota. 2025. On the Effectiveness of LLM-as-a-judge for Code Generation and Summarization. *IEEE Transactions on Software Engineering* (2025).
- [12] Edoardo Debenedetti, Giorgio Severi, Nicholas Carlini, Christopher A Choquette-Choo, Matthew Jagielski, Milad Nasr, Eric Wallace, and Florian Tramèr. 2024. Privacy side channels in machine learning systems. In *33rd USENIX Security Symposium (USENIX Security 24)*. 6848–6861.
- [13] Zehang Deng, Yongjian Guo, Changzhou Han, Wanlun Ma, Junwu Xiong, Sheng Wen, and Yang Xiang. 2025. Ai agents under threat: A survey of key security challenges and future pathways. *Comput. Surveys* 57, 7 (2025), 1–36.
- [14] Tim Dettmers and Luke Zettlemoyer. 2023. The case for 4-bit precision: k-bit inference scaling laws. In *International Conference on Machine Learning*. PMLR, 7750–7774.
- [15] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing Chat Language Models by Scaling High-quality Instructional Conversations. *arXiv:2305.14233* [cs.CL].
- [16] Ruyi Ding, Tianhong Xu, Xinyi Shen, Aidong Adam Ding, and Yunsu Fei. 2025. MoEcho: Exploiting Side-Channel Attacks to Compromise User Privacy in Mixture-of-Experts LLMs. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 2159–2173.
- [17] Yansong Gao, Huming Qiu, Zhi Zhang, Binghui Wang, Hua Ma, Alsharif Abuadba, Minhui Xue, Anmin Fu, and Surya Nepal. 2024. Deepthief: Stealing dnn model architectures through power side channel. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3311–3326.
- [18] Zibo Gao, Junjie Hu, Feng Guo, Yixin Zhang, Yinglong Han, Siyuan Liu, Haiyang Li, and Zhiqiang Lv. 2025. I Know What You Said: Unveiling Hardware Cache Side-Channels in Local Large Language Model Inference. *arXiv preprint arXiv:2505.06738* (2025).
- [19] Stefan Gast, Jonas Juffinger, Martin Schwarzl, Gururaj Saileshwar, Andreas Kogler, Simone Franza, Markus Köstl, and Daniel Gruss. 2023. Squip: Exploiting the scheduler queue contention side channel. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2256–2272.
- [20] Georgi Gerganov. 2023. llama.cpp: Inference of LLaMA models in pure C/C++. <https://github.com/ggerganov/llama.cpp>. Accessed: 2026.
- [21] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [22] Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. 2016. Flush+ flush: a fast and stealthy cache attack. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 13th International Conference, DIMVA 2016, San Sebastián, Spain, July 7-8, 2016, Proceedings 13*. Springer, 279–299.
- [23] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on llm-as-a-judge. *The Innovation* (2024).
- [24] Miro Haller. 2020. Revisiting Microarchitectural Side-Channels.
- [25] Péter Horváth, Dirk Lauret, Zhuoran Liu, and Lejla Batina. 2024. SoK: neural network extraction through physical side channels. In *Proceedings of the 33rd USENIX Conference on Security Symposium*. 3403–3422.
- [26] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. Pleak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 3600–3614.
- [27] Intel Corporation. n.d. What Is Hyper-Threading? Online. <https://www.intel.com/content/www/us/en/gaming/resources/hyper-threading.html>
- [28] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825* [cs.CL] <https://arxiv.org/abs/2310.06825>
- [29] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschiot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. *Advances in neural information processing systems* 33 (2020), 18661–18673.
- [30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [31] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2021. Deduplicating training data makes language models better. *arXiv preprint arXiv:2107.06499* (2021).
- [32] Yunxiang Li, Zihan Li, Kai Zhang, Ruilong Dan, Steve Jiang, and You Zhang. 2023. Chatdoctor: A medical chat model fine-tuned on a large language model meta-ai (llama) using medical domain knowledge. *Cureus* 15, 6 (2023).
- [33] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
- [34] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B Lee. 2015. Last-level cache side-channel attacks are practical. In *2015 IEEE symposium on security and privacy*. IEEE, 605–622.
- [35] Jialin Liu, Houman Homayoun, Chongzhou Fang, Ning Miao, and Han Wang. 2023. Side channel-assisted inference attacks on machine learning-based ecg classification. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.

- [36] Zhibo Liu, Yuanyuan Yuan, Yanzuo Chen, Sihang Hu, Tianxiang Li, and Shuai Wang. 2024. Deepcache: Revisiting cache side-channel attacks in deep neural networks executables. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 4495–4508.
- [37] Stuart Lloyd. 1982. Least squares quantization in PCM. *IEEE transactions on information theory* 28, 2 (1982), 129–137.
- [38] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
- [39] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems* 36 (2023), 21702–21720.
- [40] Deborah T Marr, Frank Binns, David L Hill, Glenn Hinton, David A Koufaty, J Alan Miller, and Michael Upton. 2002. Hyper-Threading Technology Architecture and Microarchitecture. *Intel Technology Journal* 6, 1 (2002).
- [41] Maria Mushtaq, Muhammad Asim Mukhtar, Vianney Lapotre, Muhammad Khuram Bhatti, and Guy Gogniat. 2020. Winter is here! A decade of cache-based side-channel attacks, detection & mitigation for RSA. *Information Systems* 92 (2020), 101524.
- [42] OpenClaw Foundation. [n. d.]. OpenClaw: Your own personal AI assistant. Any OS. Any Platform. <https://github.com/openclaw/openclaw>. Accessed: 2026.
- [43] Moinuddin K Qureshi. 2018. CEASER: Mitigating conflict-based cache attacks via encrypted-address and remapping. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 775–787.
- [44] Mohaimenul Azam Khan Raiaan, Md Saddam Hossain Mukta, Kaniz Fatema, Nur Mohammad Fahad, Sadman Sakib, Most Marufatul Jannat Mim, Jubaer Ahmad, Mohammed Eunus Ali, and Sami Azam. 2024. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE access* 12 (2024), 26839–26874.
- [45] Fabian Rauscher, Carina Fiedler, Andreas Kogler, and Daniel Gruss. 2025. A Systematic Evaluation of Novel and Existing Cache Side Channels. In *Network and Distributed System Security Symposium (NDSS)* 2025.
- [46] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3982–3992.
- [47] Shubhi Shukla, Manaar Alam, Sarani Bhattacharya, Pabitra Mitra, and Debdeep Mukhopadhyay. 2023. “Whispering MLaaS”: Exploiting Timing Channels to Compromise User Privacy in Deep Neural Networks. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2023), 587–613.
- [48] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. 2024. The early bird catches the leak: Unveiling timing side channels in llm serving systems. *arXiv preprint arXiv:2409.20002* (2024).
- [49] Ollama Team. 2026. Ollama Official Documentation. Online. <https://docs.ollama.com/>
- [50] Weixi Tong and Tianyi Zhang. 2024. Codejudge: Evaluating code generation with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 20032–20051.
- [51] Eran Tromer, Dag Arne Osvik, and Adi Shamir. 2010. Efficient cache attacks on AES, and countermeasures. *Journal of Cryptology* 23 (2010), 37–71.
- [52] Jo Van Bulck, Marina Minkin, Ofir Weiss, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F Wenisch, Yuval Yarom, and Raoul Strackx. 2018. Foresadow: Extracting the keys to the intel {SGX} kingdom with transient {Out-of-Order} execution. In *27th USENIX Security Symposium (USENIX Security 18)*. 991–1008.
- [53] Steven Walton. 2019. How Screwed is Intel without Hyper-Threading? Online. <https://www.techspot.com/article/1850-howscrewed-is-intel-no-hyper-threading/>
- [54] Daimeng Wang, Zhiyun Qian, Nael Abu-Ghazaleh, and Srikanth V Krishnamurthy. 2019. Papp: Prefetcher-aware prime and probe side-channel attack. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6.
- [55] Han Wang, Syed Mahbub Hafiz, Kartik Patwari, Chen-Nee Chuah, Zubair Shafiq, and Houman Homayoun. 2022. Stealthy inference attack on dnn via cache-based side-channel attacks. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1515–1520.
- [56] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [57] Roy Weiss, Daniel Ayzenshteyn, Guy Amit, and Yisroel Mirsky. 2024. What Was Your Prompt? A Remote Keylogging Attack on {AI} Assistants. In *33rd USENIX Security Symposium (USENIX Security 24)*. 3367–3384.
- [58] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clément Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 38–45.
- [59] Guanlong Wu, Zheng Zhang, Yao Zhang, Weili Wang, Jianyu Niu, Ye Wu, and Yinqian Zhang. 2025. I know what you asked: Prompt leakage via kv-cache sharing in multi-tenant llm serving. In *Proceedings of the 2025 Network and Distributed System Security (NDSS) Symposium*. San Diego, CA, USA.
- [60] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. 2025. Fast on-device LLM inference with npus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 445–462.
- [61] Mengjia Yan, Christopher W Fletcher, and Josep Torrellas. 2020. Cache telepathy: Leveraging shared resource attacks to learn {DNN} architectures. In *29th USENIX Security Symposium (USENIX Security 20)*. 2003–2020.
- [62] Yong Yang, Changjiang Li, Yi Jiang, Xi Chen, Haoyu Wang, Xuhong Zhang, Zonghui Wang, and Shouling Ji. 2024. PRSA: PROMPT Stealing Attacks against large language models. *arXiv preprint arXiv:2402.19200* (2024).
- [63] Yuval Yarom. 2016. Mastik: A micro-architectural side-channel toolkit.
- [64] Yuval Yarom and Katrina Falkner. 2014. {FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack. In *23rd USENIX security symposium (USENIX security 14)*. 719–732.
- [65] Ying Ye, Richard West, Zhuoqun Cheng, and Ye Li. 2014. Coloris: a dynamic cache partitioning system using page coloring. In *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 381–392.
- [66] Yizhen Yuan, Rui Kong, Yuanchun Li, and Yunxin Liu. 2024. Wip: An on-device llm-based approach to query privacy protection. In *Proceedings of the Workshop on Edge and Mobile Foundation Models*. 7–9.
- [67] Yuanyuan Yuan, Qi Pang, and Shuai Wang. 2022. Automated side channel analysis of media software with manifold learning. In *31st USENIX Security Symposium (USENIX security 22)*. 4419–4436.
- [68] Yuanyuan Yuan, Shuai Wang, and Junping Zhang. 2021. Private image reconstruction from system side channels using generative models. In *International Conference on Learning Representations*.
- [69] Shiquan Zhang, Ying Ma, Le Fang, Hong Jia, Simon D’Alfonso, and Vassilis Kostakos. 2024. Enabling on-device llms personalization with smartphone sensing. In *Companion of the 2024 ACM International Joint Conference on Pervasive and Ubiquitous Computing*. 186–190.
- [70] Zhiyuan Zhang, Zhenzhi Lai, and Udaya Parampalli. 2024. R+ R: Demystifying ML-Assisted Side-Channel Analysis Framework: A Case of Image Reconstruction. In *2024 Annual Computer Security Applications Conference (ACSAC)*. IEEE, 393–409.
- [71] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.

A Open Science

We use publicly available models, frameworks, and datasets throughout this work, including Phi-3-mini, Llama-3, Llama.cpp, HuggingFace Tokenizers, UltraChat, ChatDoctor, and Code-Alpaca.

We do not provide exploit artifacts or components that could directly enable misuse against real deployments. To support reproducibility, we present our code, reconstruction pipelines, and evaluation scripts at the following link: <https://github.com/royweiss1/Detokenization-Leaks>.

B Ethical Considerations

All experiments are conducted on local machines using open-source models and controlled datasets. No real user data or human subjects are involved. The identified vulnerabilities have been responsibly disclosed to relevant parties, including HuggingFace and Llama.cpp maintainers.

C Data Cleaning

To reduce redundancy in the training data, we applied an 8-gram deduplication procedure to the user prompts, following the approach adopted in PaLM and GPT-3 training [6, 8]. A training sample is removed if its prompt shares its first 8 words with any previously seen prompt, or if more than 70% of its 8-grams overlap

with those of any previously seen prompt. This deduplication criterion has also been employed in prior work [18]. Additionally, we applied further cleaning to the UltraChat dataset to remove empty or generic responses (e.g., “As an AI, I can’t help with that”). See Table 5 for dataset statistics before and after filtering.

Dataset	Before	After	Cleaned (%)
Code-Alpaca	121,959	98,540	18.8%
ChatDoctor	112,165	96,084	14.3%
UltraChat	577,747	358,545	37.9%

Table 5: Dataset sizes before and after cleaning, with percentage removed.

D LLM Prompts

Suppose R_0 = “Based on the medical information you provided, here is an explanation of the results.” and R_1 = “The report indicates that the tiredness you feel is due to stress and recommends follow-up testing.” The training inputs take the form:

Training Prompts

LLM_A (first segment)

Translate the symbolic tokens to English.

Symbolic Tokens:

$\langle \text{SYM}_{12} \rangle \langle \text{SYM}_7 \rangle \langle \text{SYM}_{31} \rangle \langle \text{SYM}_4 \rangle \langle \text{SYM}_{18} \rangle \langle \text{SYM}_{12} \rangle \langle \text{SYM}_3 \rangle \dots$

LLM_B (middle segment)

Translate the symbolic tokens to English, given the context.

Context (reconstructed):

Based on the medical **report** you **gave**, here is an explanation of the **diagnosis**.

Symbolic Tokens:

$\langle \text{SYM}_5 \rangle \langle \text{SYM}_{22} \rangle \langle \text{SYM}_9 \rangle \langle \text{SYM}_{41} \rangle \langle \text{SYM}_5 \rangle \langle \text{SYM}_{14} \rangle \langle \text{SYM}_{27} \rangle \dots$

E Additional Modeling Details

Trace preprocessing and representation. Each trace is represented as a 64-dimensional latency vector. We subtract the per-trace mean and apply feature-wise standardization over the profiling set. During training, batches are constructed by sampling token IDs and drawing multiple traces per token, producing multiple positives per token alongside cross-token negatives for contrastive learning. **Clustering and symbolization.** Following encoding, we compute a single centroid per token by averaging the embeddings of its profiling traces. We then cluster these centroids using K-means with cosine similarity. At inference, each trace is encoded and assigned to its nearest cluster center to obtain a symbolic token. This two-stage approach—encoding followed by K-means—consistently outperforms applying K-means directly to the raw traces, with particularly pronounced gains on lower-performing tokens (2× improvement on the worst-performing 10% of tokens). We attribute this to the encoder’s ability to suppress noise and amplify token-discriminative structure in the latent space.

Clustering quality. We evaluate clustering quality on held-out traces (20 test traces and 30 training traces) by measuring, for

Reconstruction Judge Prompt

System Prompt (UltraChat): “You are a privacy-leakage evaluator for reconstructed responses from side-channel observations; judge whether the prediction reveals meaningful and accurate information about the reference rather than quality, and answer Yes if it captures the same topic, intent, structure, style, or key points even if partial or noisy, otherwise No; output only Yes or No.”

System Prompt (ChatDoctor): “You are a privacy-leakage evaluator for reconstructed medical responses from side-channel observations; judge whether the prediction reveals meaningful and accurate clinical information about the reference rather than quality or safety, and answer Yes if it reflects the same specific symptoms, conditions, diagnoses, treatments, medications, reasoning, or structured advice even if partial or noisy, otherwise No; output only Yes or No”

System Prompt (Code-Alpaca): “You are a privacy-leakage evaluator for reconstructed code from side-channel observations; judge whether it reveals meaningful and accurate information about the original code rather than correctness, and answer Yes if it reflects the same general topic, structure, frameworks, APIs, algorithms, or templates even if partial or noisy, otherwise answer No; output only Yes or No.”

User Prompt:

Reference: <Original>

Prediction: <Reconstructed>

Figure 8: LLM judge prompts for reconstruction evaluation.

each token, the fraction of its traces assigned to the same cluster as its centroid, then averaging across tokens. This yields a mean clustering accuracy of approximately 75% across all trace datasets. **Reconstruction and decoding.** Training examples for LLM_B pair the preceding segment text as context with the current symbol sequence and target text. Unless stated otherwise, decoding is performed with beam search (4 beams, no sampling, maximum generation length of 38 tokens).

F K Ablation Study

We sweep $K \in \{1, 16, 32, 48, 64, 128\}$ on UltraChat (Llama.cpp, Desktop, Phi-3-mini), training a separate pipeline for each K on 100,000 training samples and evaluating on 5,000 held-out samples. As shown in Figure 9, $K = 1$ collapses to near-zero ASR, confirming the model cannot reconstruct text from an uninformative symbol alphabet. ASR rises sharply and peaks around $K = 48$ –64, then declines as larger K fragments the per-symbol training signal. $K = 48$ and $K = 64$ are within noise of each other; we use $K = 64$.

G High-Fidelity Reconstructions

While the main evaluation characterizes reconstruction quality across the full test set, we additionally examine the upper end of the fidelity distribution. Table 6 reports the fraction of reconstructions

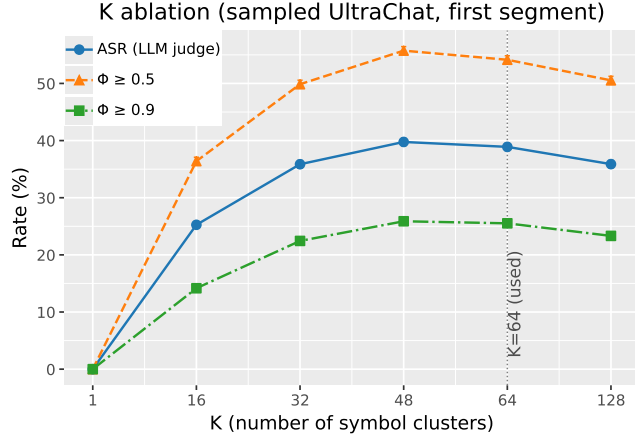


Figure 9: Reconstruction fidelity across different numbers of symbol clusters K on UltraChat first-segment reconstruction.

with high semantic similarity ($\phi \geq 0.9$), high lexical overlap ($R1 \geq 0.9$), and low normalized edit distance ($ED \leq 0.1$), together with their exact-match cases.

High-fidelity recovery is most pronounced for the first segments. For ChatDoctor, Phi-3-mini with Llama.cpp on the laptop reaches 60.43% at $ED \leq 0.1$, including 36.06% exact matches. For UltraChat, Phi-3-mini with Llama.cpp on the desktop reaches 40.43% and 18.36%, respectively. High-fidelity recovery also occurs for complete responses; for example, the same UltraChat desktop configuration achieves 24.83% at $ED \leq 0.1$. These results complement the aggregate semantic metrics by quantifying cases in which the reconstructed output closely matches the original text.

H SMT Coverage Across Intel CPUs

Our threat model (Section 3.1) assumes the victim runs on a processor exposing SMT, so that the adversary can be co-scheduled on the sibling logical processor of the core executing the victim and observe contention in the shared L1 data cache. This assumption is broadly satisfied on contemporary consumer hardware. Table 7 lists representative desktop and laptop processors across the 6th–14th Intel generations, all of which provide two logical processors per physical core. We enumerate 638 SMT-capable Intel processors released from the 6th generation (Skylake, 2015) through the 14th generation (Raptor Lake Refresh, 2024); the full list is released at our repository.

Hybrid processors. From the 12th generation onward, Intel client processors combine P-cores with E-cores, and SMT is implemented only on the P-cores, which are therefore the only cores exposing a sibling logical processor. The attack consequently requires the victim to occupy a P-core; the adversary can enumerate the sibling topology and restrict measurement to those intervals, at the cost of reduced trace yield. Our 13th generation evaluation (Section 5.5) on a hybrid Core i9-13950HX confirms that the channel persists.

Table 7: Representative Intel processors with SMT (Hyper-Threading) enabled by default, across the 6th–14th generations. † marks the platforms evaluated in this work.

Processor	Class	Generation	Cores (P+E)	Threads
Core i7-6700K	Desktop	6th	4	8
Core i5-6200U	Laptop	6th	2	4
Core i7-7700K	Desktop	7th	4	8
Core i5-7200U	Laptop	7th	2	4
Core i7-8700K	Desktop	8th	6	12
Core i5-8250U	Laptop	8th	4	8
Core i7-8650U†	Laptop	8th	4	8
Core i9-9900K	Desktop	9th	8	16
Core i7-9750H	Laptop	9th	6	12
Core i7-10700K	Desktop	10th	8	16
Core i5-10210U	Laptop	10th	4	8
Core i7-11700K†	Desktop	11th	8	16
Core i7-1165G7	Laptop	11th	4	8
Core i5-12400F	Desktop	12th	6 (6+0)	12
Core i7-12700K	Desktop	12th	12 (8+4)	20
Core i5-1235U	Laptop	12th	10 (2+8)	12
Core i5-13400F	Desktop	13th	10 (6+4)	16
Core i9-13900K	Desktop	13th	24 (8+16)	32
Core i9-13950HX†	Laptop	13th	24 (8+16)	32
Core i7-14700K	Desktop	14th	20 (8+12)	28

Figure 10: Simplified decode snippet from Llama.cpp. The highlighted line shows the token-dependent lookup.

```

1 int32_t llama_vocab::impl::token_to_piece(...) const {
2     if (!cache_token_to_piece.empty())
3         const auto & result = cache_token_to_piece.at(
4             token);
5     return _try_copy(result.data(), result.size());
6
7     if (0 <= token && token < (int32_t) id_to_token.size()) {
8         const std::string & token_text = id_to_token[
9             token].text;
10        switch (get_type()) {
11            case LLAMA_VOCAB_TYPE_SPM:
12            case LLAMA_VOCAB_TYPE_BPE: {
13                // ...
14                std::string result = ...;
15                return _try_copy(result.data(), result.size());
16            }
17            case LLAMA_VOCAB_TYPE_RWKV: {
18                std::vector<uint8_t> result = ...;
19                memcpy(buf, result.data(), result.size());
20                return (int) result.size();
21            }
22            default: {
23                std::string result = token_text;
24                return _try_copy(result.data(), result.size());
25            }
26        }
27    }
28    return 0; }

```

Table 6: Reconstruction accuracy across hardware, inference framework, target model, and segmentation configurations. Results are reported for high-fidelity reconstruction thresholds and exact matches under ϕ , ROUGE-1 (R1), and edit-distance (ED) metrics.

Dataset	Framework	Hardware	Target	First Segments						Full Paragraphs					
				$\phi \geq 0.9$	$\phi = 1$	R1 ≥ 0.9	R1 = 1	ED ≤ 0.1	ED = 0	$\phi \geq 0.9$	$\phi = 1$	R1 ≥ 0.9	R1 = 1	ED ≤ 0.1	ED = 0
UltraChat	Llama.cpp	Laptop	Phi-3-mini	41.06%	17.03%	37.65%	16.45%	37.41%	16.28%	55.38%	0.68%	22.12%	0.44%	24.01%	0.44%
			Llama-3	23.06%	5.16%	15.97%	4.85%	16.52%	4.81%	34.72%	0.13%	1.31%	0.12%	1.67%	0.12%
	HuggingFace	Desktop	Phi-3-mini	44.62%	18.93%	40.86%	18.47%	40.43%	18.36%	58.48%	0.78%	22.45%	0.51%	24.83%	0.51%
			Llama-3	28.54%	8.13%	23.49%	7.87%	23.10%	7.79%	41.66%	0.25%	4.94%	0.20%	5.60%	0.20%
		Laptop	Phi-3-mini	40.23%	13.76%	33.62%	13.16%	34.58%	13.07%	54.08%	0.38%	11.20%	0.27%	14.36%	0.27%
			Llama-3	13.42%	2.80%	8.84%	2.68%	8.95%	2.63%	21.08%	0.09%	0.60%	0.09%	0.62%	0.09%
ChatDoctor	Llama.cpp	Desktop	Phi-3-mini	38.28%	14.54%	33.64%	14.24%	33.72%	14.12%	51.84%	0.52%	15.54%	0.38%	17.63%	0.38%
			Llama-3	26.82%	6.81%	20.48%	6.40%	20.97%	6.32%	39.33%	0.20%	3.27%	0.17%	3.83%	0.17%
		Laptop	Phi-3-mini	62.92%	37.68%	58.49%	36.32%	60.43%	36.06%	16.31%	0.76%	2.36%	0.70%	3.41%	0.70%
			Llama-3	16.01%	3.69%	12.76%	3.55%	14.53%	3.47%	10.60%	0.34%	4.39%	0.25%	5.71%	0.25%
	HuggingFace	Desktop	Phi-3-mini	25.58%	0.45%	16.38%	0.28%	12.10%	0.27%	20.21%	0.09%	7.61%	0.02%	7.67%	0.02%
			Llama-3	19.95%	6.31%	17.96%	6.10%	18.97%	6.02%	13.83%	0.72%	8.19%	0.46%	9.19%	0.46%
		Laptop	Phi-3-mini	27.78%	6.67%	20.75%	6.42%	23.35%	6.19%	23.27%	0.62%	11.07%	0.32%	13.50%	0.32%
			Llama-3	10.81%	2.88%	8.94%	2.78%	9.62%	2.73%	5.40%	0.25%	2.66%	0.20%	2.99%	0.20%
		Desktop	Phi-3-mini	30.28%	11.18%	26.85%	11.11%	27.66%	10.72%	24.77%	1.69%	15.22%	0.99%	17.20%	0.99%
			Llama-3	19.45%	5.52%	16.48%	5.13%	17.91%	5.02%	13.17%	0.51%	7.40%	0.32%	8.85%	0.32%

Figure 11: Simplified snippet of the decode path in the HuggingFace Tokenizers, which is used by Transformers. The highlighted line shows the token-dependent lookup.

```
1 pub fn decode(&self, ids: &[u32], skip_special_tokens:
    bool) -> Result<String> {
2     let tokens = ids
3         .iter()
4         .filter_map(|id| {
5             self.added_vocabulary
6                 .simple_id_to_token(*id)
7                 .or_else(|| self.model.id_to_token(*id))
8                 .filter(|token| {
9                     !skip_special_tokens || !self.
10                        added_vocabulary.
11                        is_special_token(token)
12                })
13        })
14        .collect::<Vec<_>>();
15
16 if let Some(decoder) = &self.decoder
17     decoder.decode(tokens)
18 else
19     Ok(tokens.join(" "))
20
21 fn id_to_token(&self, id: u32) -> Option<String> {
22     self.vocab_r.get(&id).cloned() }
```

Attack Examples – Code Reconstruction

Judge: Success

JavaScript (GraphQL)

```
Ref: const ApolloServer, gql = require('apollo-server');
const typeDefs = gql{ type Product { id: ID!, name: String!,
description: String, price: Float! } type Query {
product(id: ID!): Product } }; const products = { '123': {
id: '123', name: 'T-Shirt', description: 'This is a sample
product', price: 29.99 } }; const resolvers = { Query: {
product: (_parent, { id }) => products[id] } }; const server
= new ApolloServer({ typeDefs, resolvers });
server.listen().then(({ url }) => console.log(url));

Pred: const ApolloServer, gql = require('apollo-server');
const typeDefs = gql{ type Product { id: ID!, name: String!,
price: Int! } type Query { products: [Product], product(id:
ID!): Product } }; const products = { '456': { name: 'Jane
Doe', description: 'Alice in Wonderland', price: 10.99 } };
const resolvers = { Query: { products: () => products } };
const server = new ApolloServer({ typeDefs, resolvers });
server.listen().then(({ url }) => console.log(url));
```

Judge: Success

JavaScript (Express/MySQL)

```
Ref: const express = require('express'); const mysql =
require('mysql'); const app = express(); const db =
mysql.createConnection({ host: 'localhost', user: 'user',
password: 'pass', database: 'db' }); db.connect();
app.get('/customers', (req, res) => { db.query("SELECT *
FROM Customers", (err, results) => { if (err) throw err;
res.send(results); }); }); app.post('/customers', (req, res)
=> { const { name, email } = req.body; db.query("INSERT INTO
Customers (name,email) VALUES (?,?)", [name,email]); });

Pred: const express = require('express'); const mysql =
require('mysql'); const app = express(); const db =
mysql.createConnection({ user: 'root', password: '' });
db.connect(); app.get('/customers', (req, res) => {
db.collection('customers').find(); });
app.post('/customers', (req, res) => { const { name, email }
= req.body; db.query("INSERT Customers (name,email)",
[name]); });
```

Judge: Success

C# (AES encryption)

